Project 1 Load flow study

1st Andreas Pedersen Monsen 2nd Sverre Prøsch-Moen 3rd Tor Kristian Boiesen

4th Håkon Nordal Grenne 5th Theo Bråten 6th Vegard Ernstsen Gresslien

I. Introduction

This study is the first part of a course project in TET4205 Power system analysis 2. The goal of this study is to develop an in-house solver and validate it with an open-source solver. Both solvers are applied to conduct a load flow study on the regional power system in Trøndelag, Norway. The solvers are however designed to handle any network, given the proper excel structure.

The implemented load flow solvers is as follows: Newton-Raphson (NR), Fast decoupled load flow (FDLF) and DC power flow (DCPF). The in-house solvers will be validated with the open-source solver: Pandapower. Data handling and computations is all done through Python.

II. Case study and data

The chosen power system network is a modified model of the transmission grid in the Norwegian bidding zone NO3. The power system grid stretches from Namsos in Trøndelag county and all the way to Svelgen in Vestland county. The model consist of three voltage levels : 132 kV, 300 kV and 380 kV. The power system is coupled together by 22 lines and 8 transformers, and includes a total of 104 generators which supplies a total of 3.9 GW of hydro power and 0.3 GW of thermal power. The overview of the power system is shown in figure 1.



Fig. 1. Bidding area NO3

The system presented in Table II, contains 25 buses: one slackbus, 16 PV buses and 8 PQ buses. The power system has been modified to be applicable for a N-1 criteria-test in accordance to the project description. PQ-bus 177-180 is therefore added. The single-line diagram (SLD) of the modified network, including voltage magnitude and phase angle heatmaps, is presented in the Appendix 2.

The power base (S_{base}) is chosen to be 100 MVA and the global voltage base ($V_{base,global}$) is defined by the slack bus to be 300 kV. With the selected power base and global voltage base known, the system parameters was converted into per-unit values to conduct the study.

1) Data: System network data used to conduct the power flow study is extracted from a public GitHub repository [1]. The GitHub repository imports the Nordic power grid from Ensto-E and NordPool from the year 2018.

III. Software implementation

A. Software and methodology

The power flow software was developed in Python using a modular, object-oriented structure. Network data are imported from Excel using the `MakeGrid` function and organized into a `grid` object containing buses, transmission lines, transformers and generators. The data processing includes component linking, per-unit conversion and initialization of the network parameters.

The bus admittance matrix $\mathbf{Y}$ is assembled with Van Cutsem's algorithm, following the course lectures [2]. Each line and transformer contributes a 2×2 branch matrix, which makes the assembly independent of network size and topology. The resulting network model is used by three separately implemented load flow solvers: Newton-Raphson (NRLF), Fast Decoupled Load Flow (FDLF) and DC Power Flow (DCPF). The solvers are integrated into the 'Grid' class, providing a common interface for executing the analyses.

B. Newton-raphson load flow

The Newton Rhapson (NR) solver is generic, and formulated in polar coordinates. But in this case, 0 is selected as slack. Buses with generator are assigned as PV and the remaining buses are assigned PQ. Buses with a all zero row in $\mathbf{Y}$ are marked as isolated, and excluded from further calculations. The unknowns are ordered bus by bus, with δ for PV buses and δ, V for PQ buses. The Jacobian is assembled element by element from the analytical derivatives as a dense NumPy array, and rebuilt in every iteration and solved with `numpy.linalg.solve`. Reactive generator limits are checked in each iteration by testing if a PV bus violates

Q_{min} or Q_{max}, one bus at a time is converted to PQ with Q fixed at the violated limit. After convergence a limited bus is released back to PV if its voltage lies on the wrong side of its setpoint, with a cap on the number of releases to prevent oscillation. The solver stops when the largest mismatch is below 10^{-6}.

C. Fast decoupled load flow

The FDLF solver implemented is generic and exploits the weak coupling between P–δ and Q–$|V|$ in transmission networks($X \gg R$, small angle differences, $|V| \approx 1$ pu). It operates on the same grid object built for the NR solver. Bus 0 is assigned as slack, buses with generators as PV and the remaining buses are assigned PQ. $\mathbf{B}'$ is assembled from $1/X$ of all lines and transformers for the non-slack buses, while $\mathbf{B}''$ is extracted as $-\mathrm{Im}(\mathbf{Y})$ of the PQ buses from the admittance matrix shared with NR. Both $\mathbf{B}'$ and $\mathbf{B}''$ are NumPy arrays built only once. In each iteration, $\mathbf{B}'\Delta\boldsymbol{\delta} = \Delta\mathbf{P}/\mathbf{V}$ is solved with `numpy.linalg.solve` and angles are updated. Afterwards $\Delta\mathbf{Q}$ is then recalculated with the updated angles, and $\mathbf{B}''\Delta\mathbf{V} = \Delta\mathbf{Q}/\mathbf{V}$ is solved for the PQ-bus voltage magnitudes. The solver stops when the largest ΔP falls below 10^{-6} pu, or after 100 iterations. Results are stored in the solution object used by all solvers. Reactive generator limits aren't enforced in FDLF.

D. DC Power flow

DC power flow is generic and non-iterative. The susceptance matrix $\mathbf{B}$ is assembled from $1/X$ of all lines and transformers. The transformer branches are scaled by their off-nominal tap ratio per unit, and $X = 0$ branches are rejected. Bus 0 is assigned as slack and together with isolated buses removed before $\mathbf{B}\boldsymbol{\delta} = \mathbf{P}$ is solved with `numpy.linalg.solve`. Line flows are calculated as $P_{ij} = (\delta_i - \delta_j)/X_{ij}$ and net injections are calculated as $\mathbf{B}\boldsymbol{\delta}$. Voltage magnitudes are set to 1 pu and reactive power isn't computed. The solution is accepted if all angles are finite and the residual is below 10^{-6} pu.

E. Validation

The implemented load flow solvers were validated by comparing their results with corresponding solutions obtained from Pandapower, an open-source Python library for power system analysis. A separate validation script was used for the comparison. For the NR and FDLF methods, the validation focused on bus voltage magnitudes and voltage angles, as these are the main state variables in an AC load flow. The validation of DCPF focused on voltage angles, since the DC formulation assumes constant voltage magnitudes and neglects reactive power.

To ensure a consistent comparison, the same modelling assumptions and input data were reproduced for the Pandapower solvers. An absolute tolerance of 10^{-3} was applied to voltage magnitudes and angles, with the relative tolerance set to zero. The tolerance was chosen to allow small numerical differences between the implementations while maintaining a strict comparison of the calculated state variables. All three tests passed.

Small differences between the NR implementations shown in table III and IV may be explained by how generator reactive power limits are handled. In the implemented NR solver, a bus can switch from PV to PQ when a reactive power limit is reached, but it can also switch back to PV if the limit is no longer required after convergence. Pandapower also enforces reactive power limits, but does not use the same explicit mechanism for switching a bus back to PV. This difference may result in different final bus classifications and consequently small deviations in voltage magnitudes, angles and power flows, even when both solvers converge successfully.

F. Bug test

1) Input data pipeline: To verify that the input data is correctly imported, two tests were made. Test 1 verifies that the Excel file is structured as the Python program expects. It checks for missing or extra sheets, and that each sheet contains the expected column headers in the correct order. This test was chosen because the import reads data by column name, and deviations in the input file caused data to be silently read as missing. Test 2 verifies that the data from the Excel file is completely imported into the grid object created by MakeGrid. The test goes through every bus, line, transformer and generator and checks that all attributes required by the solvers are present and not empty, and reports which object and attribute is missing. This test was chosen because missing attributes otherwise surfaced as unclear errors inside the solvers, far from their actual cause.

2) Validating line impedance: The bug test first checks whether any line impedances in the data sheet equal zero, and reports any lines where this is the case. Secondly, it checks whether any line impedances are very close to zero. This test was necessary because the line admittances are calculated by $y = \frac{1}{Z}$, a zero impedance causes division by zero, while a near-zero impedance gives extremely large admittances, making the results invalid.

G. Edge cases

1) Divergence handling test: This test was implemented with the intention of checking if the NR solver handles divergence with very large values. Since the NR solver uses `numpy.linalg`, to find ΔX, this might lead to the code failing internally, and not giving the correct output of non-convergence. Therefore the divergence handling test was implemented, checking if the NR solver returns that convergence is `False` when given load values 10^4 higher than the usual network.

2) No-load test: This test was implemented to verify if the code still runs as expected when load values are set to zero. The test receives two identical networks. Then it changes all P_{load} and Q_{load} values to zero in the first network, and 10^{-4} pu in the next. Each solver is then run on both networks, and the test asserts that they give the same convergence result and.
3) No-generator test: This test was implemented to observe if the code still runs as expected when generator values are set to zero. It functions identically to the no-load test, except that the P_{gen} and Q_{gen} values were changed instead. It checks if both both solutions of the chosen solver converges.

H. Functional tests

Three functional tests run NR, FDLF and DCPF on the Trøndelag case, and assert that each solver reports convergence. They were chosen as a minimal check that every solver runs end to end on the reference network after code changes. Since they only check the convergence flag, solution accuracy is verified separatley by the validation tests against pandapower III-E

IV. Load flow studies

A. Contingency analysis

To ensure that the power system is operated securely, a contingency analysis is performed according to the N-1 criterion. The criterion states that the system shall withstand the outage of any single component, such as a line, transformer or generator, without interruption of supply to consumers and without violating operational limits on bus voltages and branch loadings. To conduct a contingency analysis it was necessary to add more power lines and transformers to make the grid properly meshed. Power lines were created by adding more transformers and lines in the excel sheets. The reason for adding more transformers is because the radial parts have different voltages to each other. The power lines are added by using longitudes and latitudes and measuring the distance between the buses.

To simulate the N-1 criterion, a three-step Python script was developed (`N_minus_en.py`). First, each line is removed in turn to check whether the remaining lines and transformers still connect all buses. A line whose removal splits the grid into isolated islands is excluded from further testing, as the load flow solvers cannot converge for an islanded system. In this case study, no single line outage creates an island, so all lines were included in the analysis.

Second, every contingency is simulated by removing one line at a time. For each case, the grid model is rebuilt from the input data and solved with all in-house solvers (NR, FDLF and DCPF) under identical conditions.

Finally, each solution is checked against a set of operational limits: bus voltages within V_{min} and V_{max}, apparent power on buses and lines against S_{max}, and generator output against P_{max}, Q_{max} and Q_{min}.

The contingency analysis showed that the power system operates accordingly to the N-1 criterion and within the operational limits. Since the grid were modified the power system cannot create any isolated islands, and therefore converges in all cases.

V. Profiling

A. Test System and Timing Results

The solvers were tested on the Nordic power system imported from `Nordic490_komplett.xlsx`, containing 500 buses, of which 497 were active. Each solver was executed three times, and the median execution times are shown in Table I.

TABLE I
Median execution times for the Nordic test system.

Solver	Time [s]
Own DCPF	0.165
Pandapower DC	0.401
Own FDLF	45.046
Pandapower FDLF	8.492

Our DCPF is approximately 2.4 times faster than the measured pandapower DC workflow, whereas our FDLF is approximately 5.3 times slower than the pandapower FDLF workflow. However, network construction is included in the pandapower timings but excluded from the timing of our own solvers, so these are not direct comparisons of solver execution alone.

B. Bottlenecks and Theoretical Discussion

For our DCPF, profiling identifies bus classification and line flow calculations as the main bottlenecks. Its short runtime is consistent with theory, since DCPF uses a linearized model and avoids iterative nonlinear AC load flow calculations.

For our FDLF, the main bottleneck is the repeated calculation of active and reactive power mismatches in `finalsolver()`. Nested loops evaluate all bus pairs, giving approximately $O(N^2)$ calculations per mismatch evaluation. The functions `flyt_P()`, `flyt_Q()`, and `numpy.angle()` are called millions of times, while the 18 linear system solves take only about 0.046 seconds cumulatively. The runtime is therefore dominated by repeated power flow calculations rather than the linear solves.

This differs from the theoretical motivation of FDLF, in which the constant matrices $\mathbf{B}'$ and $\mathbf{B}''$ make each iteration inexpensive. Vectorizing the mismatch evaluation would be expected to reduce the runtime substantially. This was identified but not implemented due to time constraints, so the FDLF time in Table I reflects our implementation, not FDLF as a method.

For pandapower, profiling shows that network construction and pandas data-frame operations dominate the measured workflows, particularly for FDLF. Thus, the timings reflect preprocessing and implementation overhead as well as solver performance. Overall, the profiling illustrates the importance of efficient implementation when evaluating the computational advantages of DCPF and FDLF.

VI. Conclusion

This project presented the development and validation of three self-built power flow solvers: Newton-Raphson (NR), Fast Decoupled Load Flow (FDLF), and DC Power Flow (DCPF). These solvers were implemented in python using a network model and validated against corresponding implementations in Pandapower. All three methods successfully passed the validation test within the specified tolerance of (10^{-3}). This demonstrates that the developed software produces consistent load flow results.

The contingency analysis demonstrated that the modified 25-bus power system satisfied the N-1 criterion for the investigated single-line outages. All simulated cases converged without violating the evaluated operational limits, indicating that the additional network connections improved the system's ability to withstand individual line outages.

The performance analysis highlighted significant differences in computational efficiency between the implemented methods. DCPF demonstrated the shortest execution time, while profiling revealed that repeated power mismatch calculations were the main bottleneck in the FDLF implementation. These findings emphasize that computational performance depends not only on the mathematical method, but also on implementation efficiency.

Overall, the project objectives of developing, applying, and validating independent load flow solvers were achieved. The developed framework provides a foundation for further power system analysis, including contingency studies. Future improvements could focus on optimizing the FDLF, reducing computational overhead, and extending the analysis of the modified network under N-1 operating conditions.

References

[1] iasonask, "spatial-disaggregation: New open source model of the Nordic power system for the EU project Spine," Master thesis project, GitHub repository. [Online]. Available: https://github.com/iasonask/spatial-disaggregation (accessed Oct. 13, 2026).

[2] V. V. Vadlamudi, "TET4205 – Power System Analysis 2: Building of Network Matrices – Focus on YBus," lecture slides, Dept. of Electric Energy, Norwegian University of Science and Technology (NTNU), Trondheim, Norway, Autumn 2023.

VII. Appendix

TABLE II
BUS CLASSIFICATION OF THE CASE STUDY NETWORK.

Bus ID	Name	Type
30	Vågåmo	Slack
45	Aura	PV
62	Brattset	PV
76	Klæbu	PV
77	Thamshavn	PV
79	Grytten	PV
80	Brattset	PV
105	Verdal	PV
106	Strinda	PV
107	Grana	PV
108	Nea	PV
109	Ørskog	PV
138	Ogndal	PV
139	Eidum	PV
141	Ørskog	PV
161	Svelgen	PV
176	Namsos	PV
46	Brattset	PQ
60	Orkdal	PQ
61	Viklandet	PQ
78	Klæbu	PQ
177	Svelgen	PQ
178	Brattset	PQ
179	Brattset	PQ
180	Namsos	PQ

TABLE V
NUMBER OF ITERATIONS

Method	Iterations
NR	4
DC	1
FD	8
PP_NR	2
PP_DC	0
PP_FD	8

TABLE VI
COMPUTATION TIME

Method	Runtime [s]
NR	0.122489
DC	0.002063
FD	0.078297
PP_NR	0.602149
PP_DC	0.356047
PP_FD	0.426931

VIII. Individual Contributions

The individual contributions to the project are summarized below. The contributions are based on the work performed by each group member and the corresponding development history.

- **Andreas Pedersen Monsen:** [Responsible for programming and implementation of DC-Pandapower, and project writing.]
- **Sverre Prøsch-Moen:** [Responsible for implementation of DCPF, Single line diagram, and voltage magnitude- and angle- heatmaps, and project writing.]
- **Tor Kristian Boiesen:** [Responsible for programming the Newton–Raphson method and designing the class structure used in the project.]
- **Håkon Nordal Grenne:** Responsible for implementation of pandapower FDLF and profiling as well as analysis of the profiling results.
- **Theo Bråten:** [Responsible for programming of FDLF and edge-case tests. Responsible for writing about implementation of NR, FDLF and DCPF as well as edge cases and functional tests in chapter 3.]
- **Vegard Ernstsen Gresslien:** NR with Pandapower solver, Validation for all three solver methods and report writing.

TABLE III
BUS VOLTAGE MAGNITUDES [PU] FOR EACH LOAD FLOW METHOD.

Bus ID	Own implementation			pandapower		
	NR	DC	FD	NR	DC	FD
30	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
45	1.001594	1.000000	1.000000	1.002371	1.000000	1.000000
46	1.003272	1.000000	0.997357	1.003434	1.000000	0.997357
60	1.004162	1.000000	1.001685	1.004411	1.000000	1.001685
61	1.009226	1.000000	1.002830	1.011253	1.000000	1.002830
62	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
76	1.003229	1.000000	1.000000	1.003384	1.000000	1.000000
77	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
78	1.018442	1.000000	1.002596	1.019374	1.000000	1.002596
79	1.004860	1.000000	1.000000	1.007485	1.000000	1.000000
80	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
105	1.000481	1.000000	1.000000	1.000599	1.000000	1.000000
106	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
107	1.010513	1.000000	1.000000	1.010514	1.000000	1.000000
108	1.025675	1.000000	1.000000	1.026378	1.000000	1.000000
109	1.001891	1.000000	1.000000	1.005110	1.000000	1.000000
138	1.002186	1.000000	1.000000	1.002363	1.000000	1.000000
139	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
141	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
161	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000
176	1.008359	1.000000	1.000000	1.008657	1.000000	1.000000
177	1.011916	1.000000	1.011922	1.011916	1.000000	1.011922
178	1.003965	1.000000	1.002143	1.004015	1.000000	1.002144
179	1.020847	1.000000	1.001935	1.021365	1.000000	1.001935
180	1.021158	1.000000	1.003765	1.021668	1.000000	1.003765

TABLE IV
BUS VOLTAGE ANGLES [RAD] FOR EACH LOAD FLOW METHOD.

Bus ID	Own implementation			pandapower		
	NR	DC	FD	NR	DC	FD
30	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
45	0.026399	0.029481	0.026190	0.026271	0.029481	0.026190
46	0.090296	0.095914	0.092298	0.090245	0.095914	0.092298
60	-0.001172	0.004198	-0.001630	-0.001182	0.004198	-0.001630
61	0.021485	0.026690	0.021086	0.021345	0.026690	0.021086
62	0.124831	0.129141	0.125847	0.124808	0.129141	0.125847
76	-0.015250	-0.009854	-0.015739	-0.015237	-0.009854	-0.015739
77	0.031121	0.038494	0.030648	0.031106	0.038494	0.030648
78	0.039587	0.046096	0.040462	0.039526	0.046096	0.040462
79	0.015409	0.020574	0.014773	0.015224	0.020574	0.014773
80	0.161647	0.165014	0.162641	0.161624	0.165014	0.162641
105	-0.032939	-0.027642	-0.034021	-0.032912	-0.027642	-0.034021
106	-0.027133	-0.021984	-0.028171	-0.027094	-0.021984	-0.028171
107	0.167906	0.174044	0.171007	0.167887	0.174044	0.171007
108	0.071511	0.079577	0.074821	0.071427	0.079577	0.074821
109	-0.000443	0.004962	-0.001528	-0.000609	0.004962	-0.001528
138	-0.021868	-0.016581	-0.022737	-0.021850	-0.016581	-0.022737
139	-0.026869	-0.021746	-0.027955	-0.026828	-0.021746	-0.027955
141	-0.067118	-0.059956	-0.068279	-0.067013	-0.059956	-0.068279
161	0.195533	0.197178	0.195153	0.195567	0.197178	0.195153
176	-0.000969	0.004837	-0.000834	-0.000974	0.004837	-0.000834
177	0.123222	0.124438	0.122981	0.123244	0.124438	0.122981
178	0.027784	0.029499	0.028453	0.027767	0.029499	0.028453
179	0.078211	0.085211	0.080707	0.078148	0.085211	0.080707
180	0.036255	0.043063	0.037267	0.036226	0.043063	0.037267

IX. SLD WITH VOLTAGE MAGNITUDE- AND ANGLE-HEATMAP

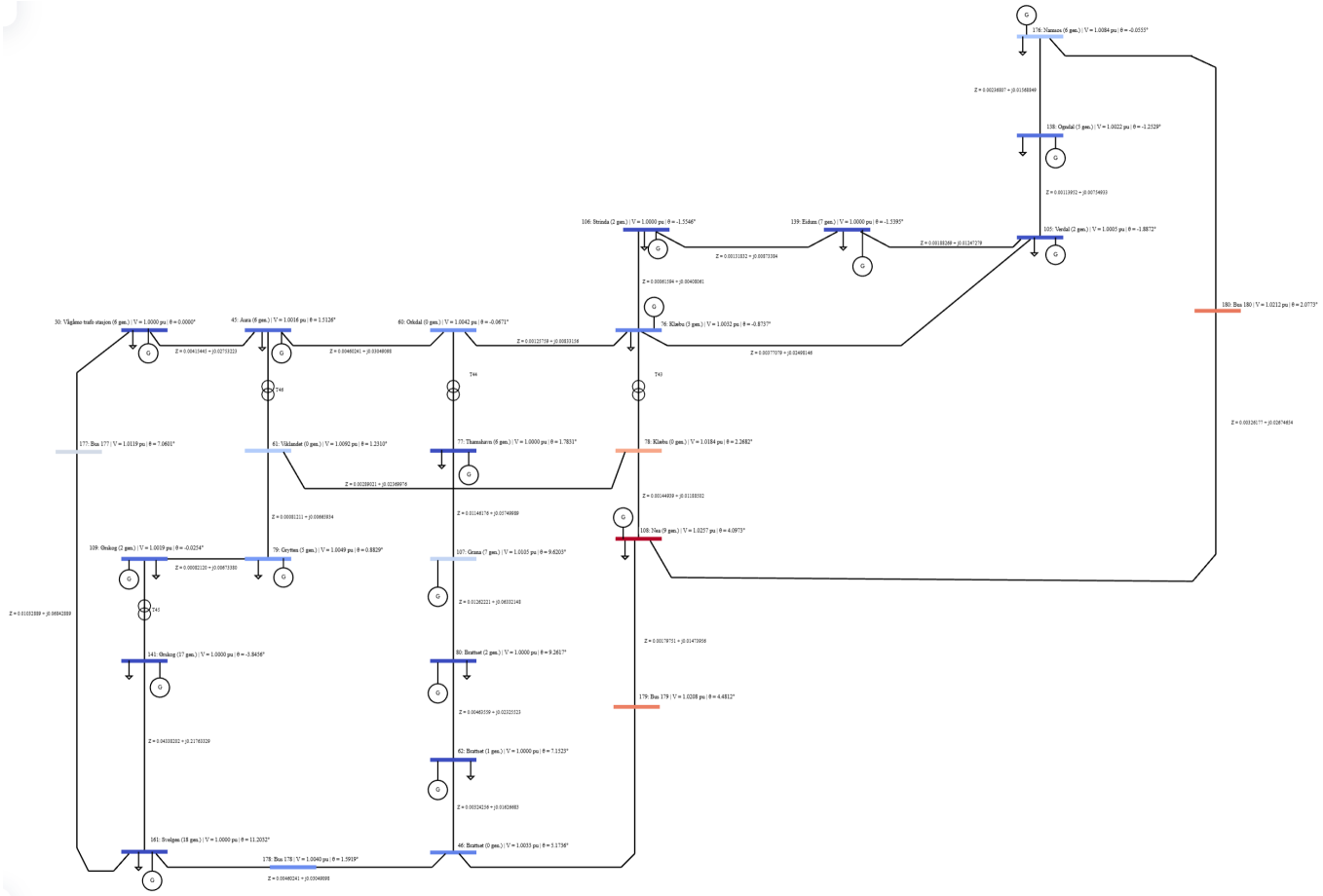


Fig. 2. Single-line diagram of the modified power system, including voltage magnitude and phase angle heatmaps.

The single-line diagram (SLD) illustrates the modified power system used in the load flow study, consisting of 25 buses interconnected by transmission lines and transformers. The diagram also shows the connected generators and transmission line impedances in per unit.

The voltage magnitude and phase angle at each bus are obtained from the Newton-Raphson load flow solution and presented numerically alongside the bus labels. The buses are colour-coded to visualise the voltage profiles across the network, with separate heatmaps for voltage magnitude and phase angle. In both heatmaps, blue represents the lowest values, while red represents the highest values within the corresponding range. This provides a visual overview of voltage variations and phase angle differences throughout the system.

X. Use of Artificial Intelligence

AI tools were used to support selected parts of the project, including coding and report work. The tools were used for code assistance, debugging, and improving report structure and Latex formatting. AI was also helpful in our struggles with Github, as we had some Github conflicts that AI helped solve. The suggestions were reviewed by the group, and relevant results were checked against the implementation, test results, and course theory. The group remains responsible for the correctness of the code, results, and final report.